

CST-407 Activity 4 Secure Network Communication

Alex M. Frear

College of Science, Engineering, and Technology, Grand Canyon University

Course Number: CST-407

Professor: Dr. Melody White

08/11/2025

Table of Contents

Build the Login Application	3
Screenshots	3
Login Page Loaded (App Running).....	3
Successful Login Attempt	4
Failed Login Attempt	5
Test the Application	6
Screenshots	6
Wireshark Capture – Successful Login Attempt	6
Wireshark Capture – Failed Login Attempt	7
Wireshark – GET / (200 OK) Details.....	8
Wireshark – Follow HTTP Stream (GET)	9
Wireshark – POST /login Form Data.....	10
Wireshark – Follow HTTP Stream (POST)	11
Transitioning to Secure HTTPS Communication	12
Screenshots	12
Browser – HTTPS on 8443 (Warning)	12
Browser – Certificate Details (Self-Signed).....	13
Wireshark – TLS Capture (HTTPS Login)	14
Summary of Key Concepts	15
Other Common Protocols	16

Build the Login Application

Screenshots

Login Page Loaded (App Running)

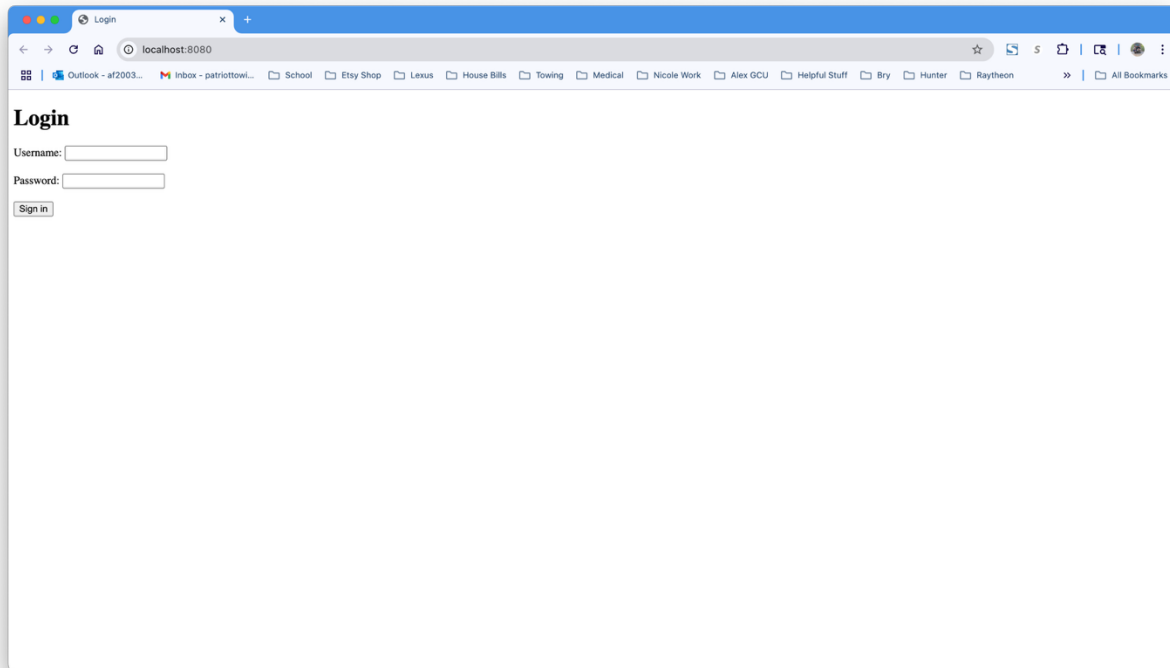


Figure 1 Browser showing `http://localhost:8080/login` form served by the Spring Boot app.

Successful Login Attempt

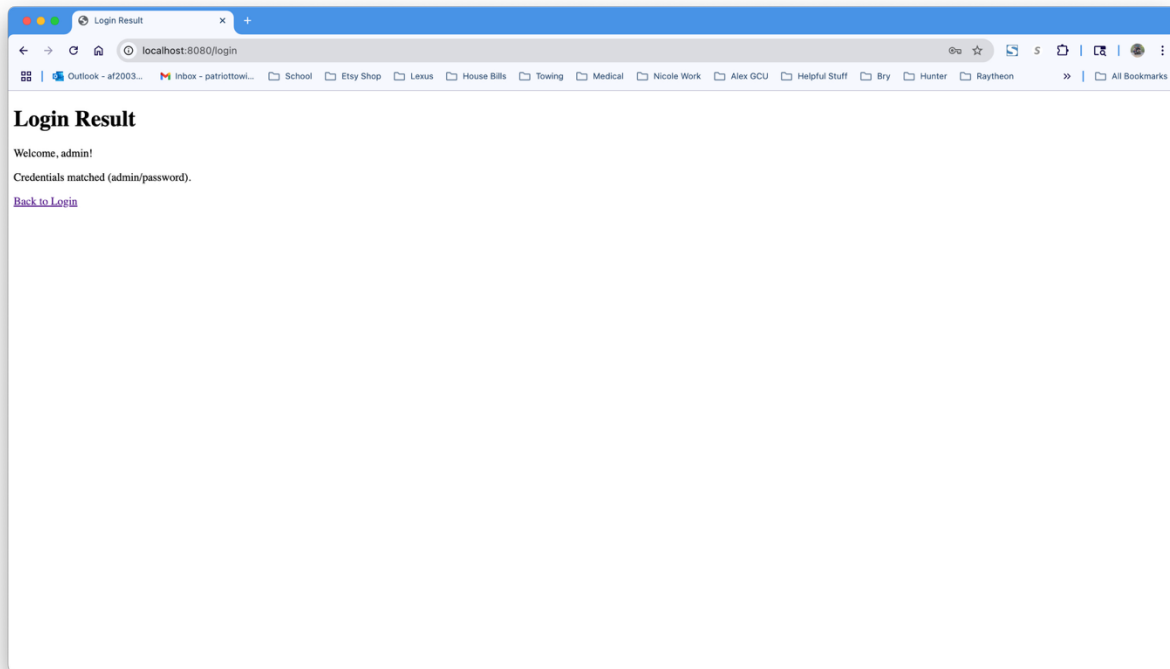


Figure 2 Browser showing successful login message after entering valid credentials (admin / password).

Failed Login Attempt

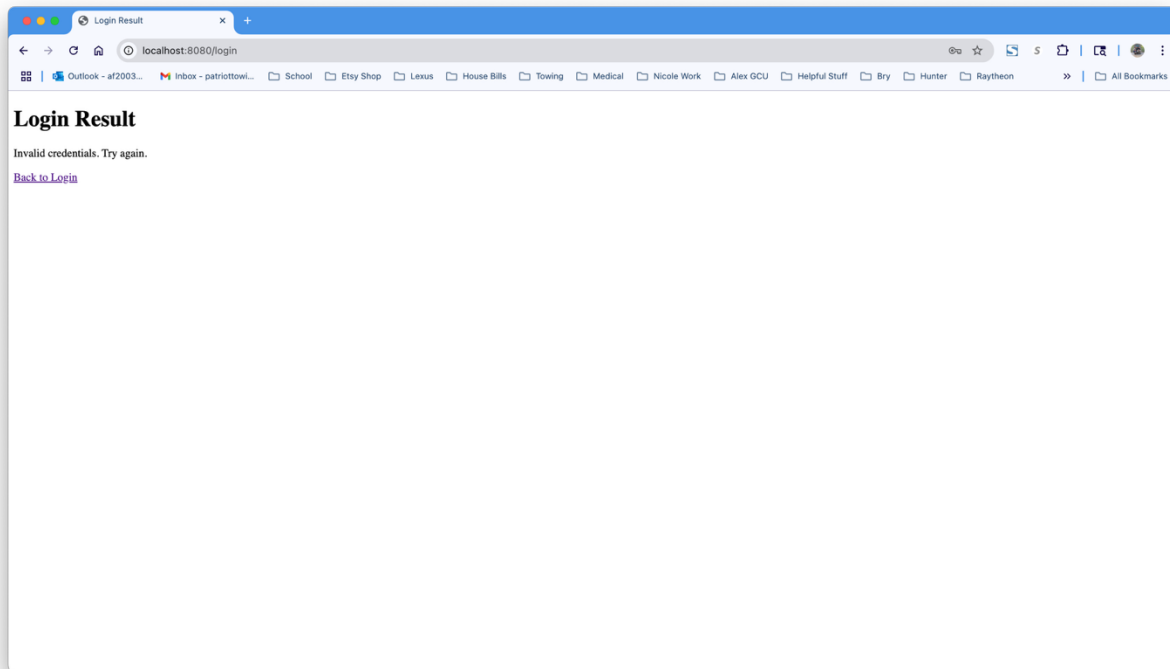


Figure 3 Browser showing error message after entering invalid credentials.

Test the Application

Screenshots

Wireshark Capture – Successful Login Attempt

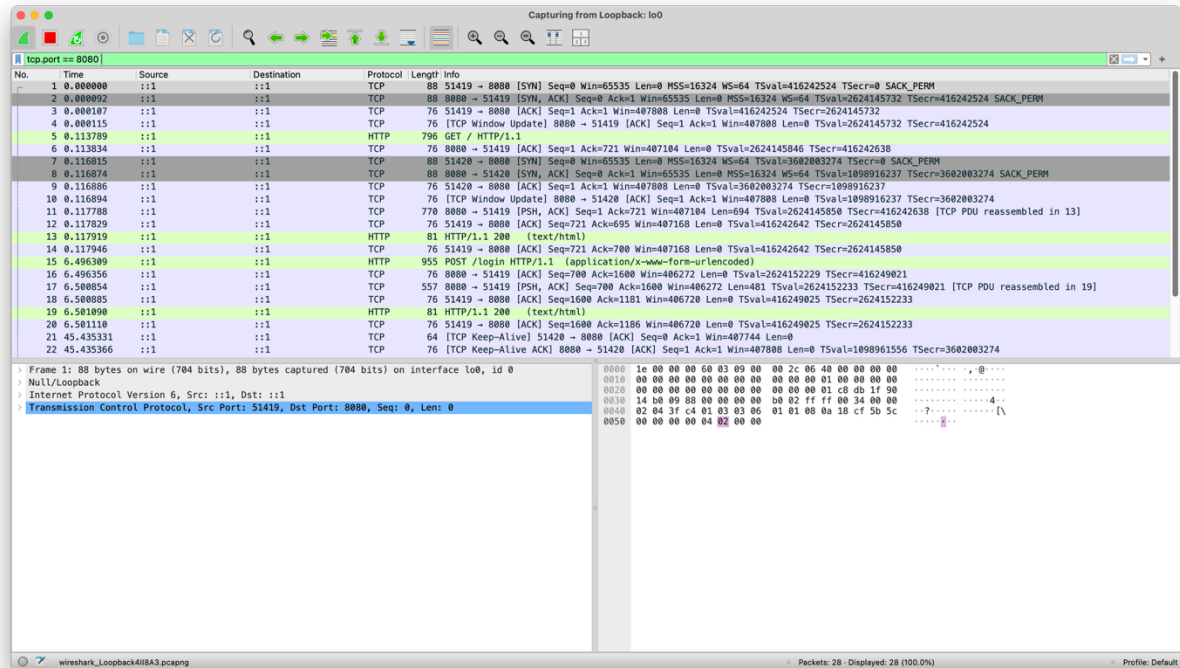


Figure 4 Wireshark packet capture showing HTTP POST request during successful login attempt.

Wireshark Capture – Failed Login Attempt

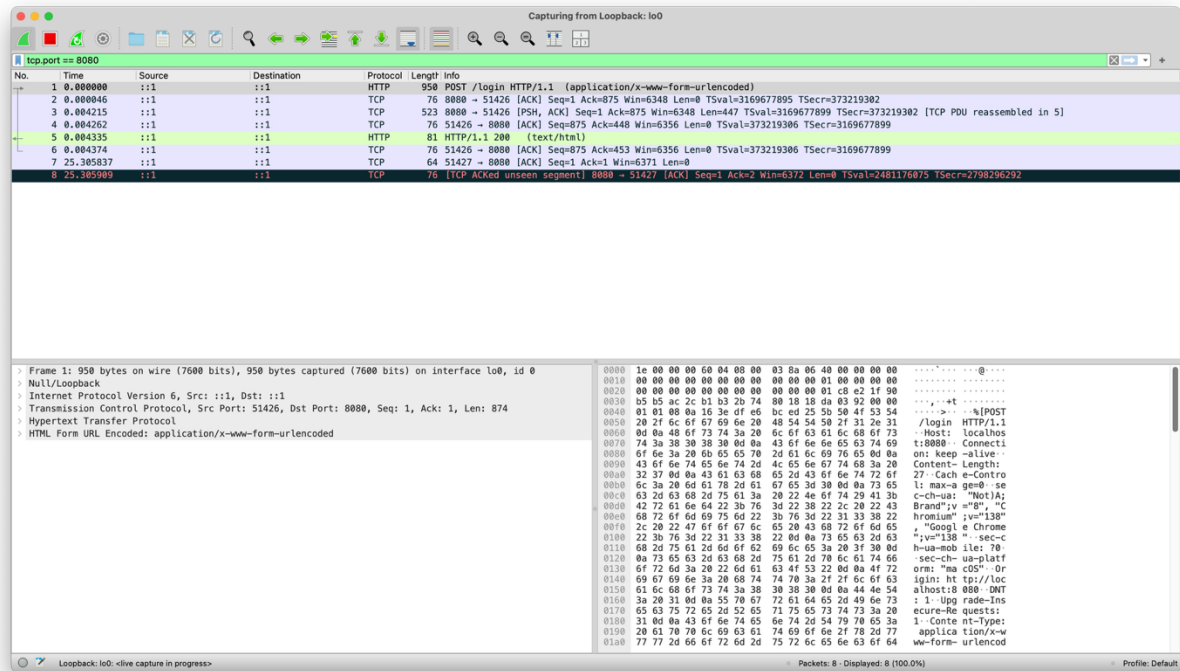


Figure 5 Wireshark packet capture showing HTTP POST request during failed login attempt.

Wireshark – GET / (200 OK) Details

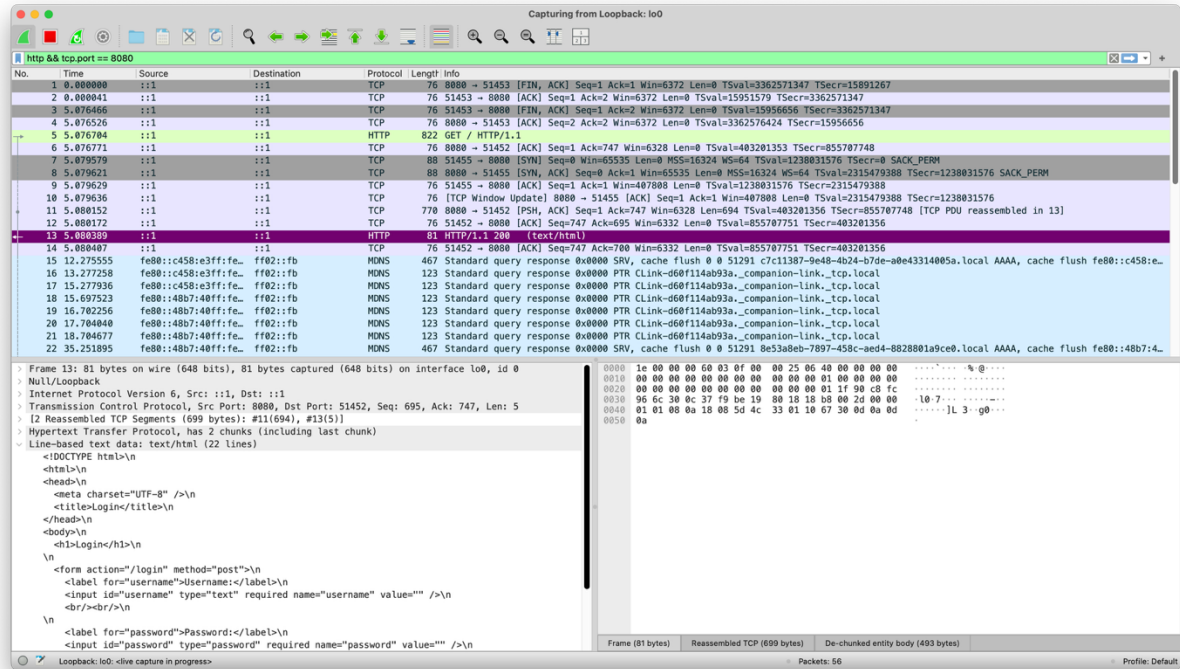


Figure 6 HTTP response to GET "/" expanded to show "Line-based text data" (login page HTML).

Wireshark – Follow HTTP Stream (GET)

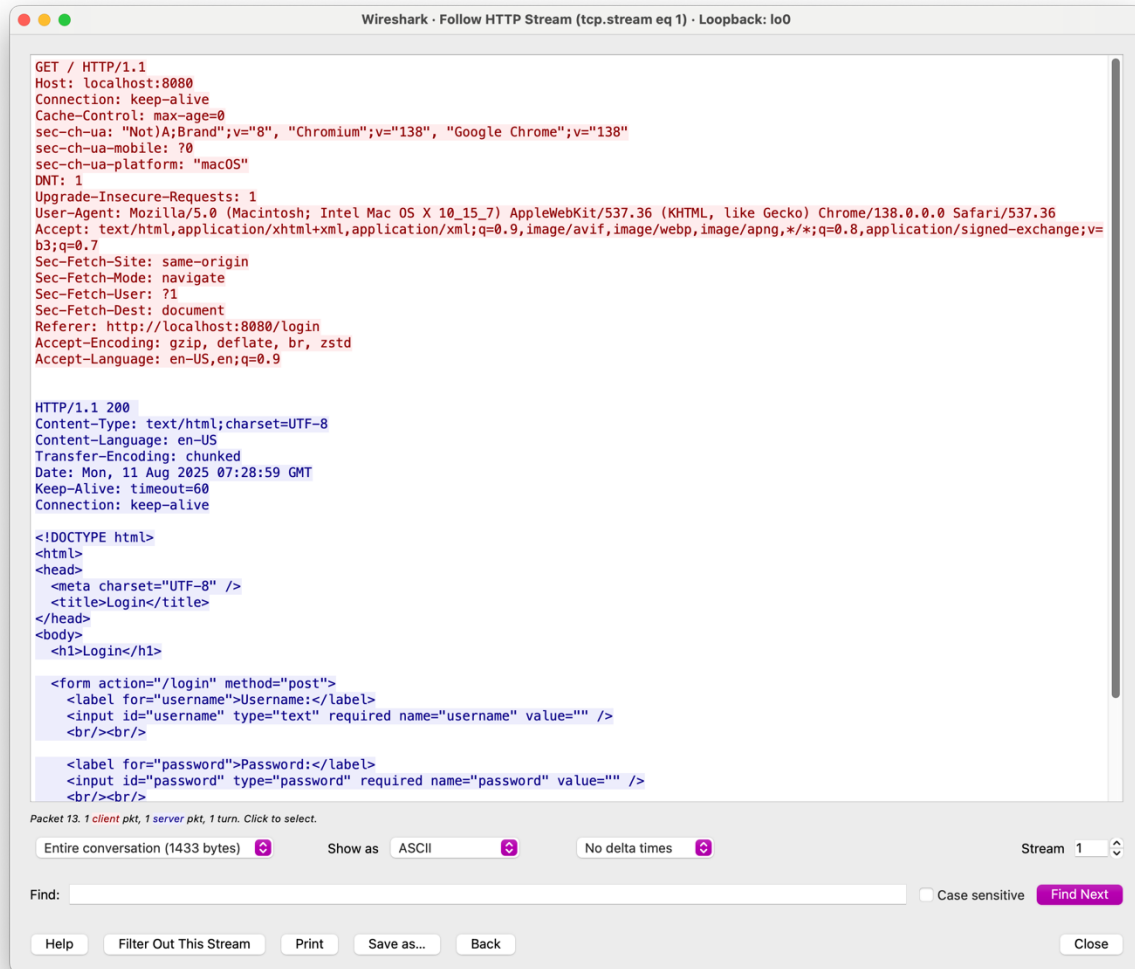


Figure 7 Followed GET stream showing HTTP request headers and HTML page content.

Wireshark – POST /login Form Data

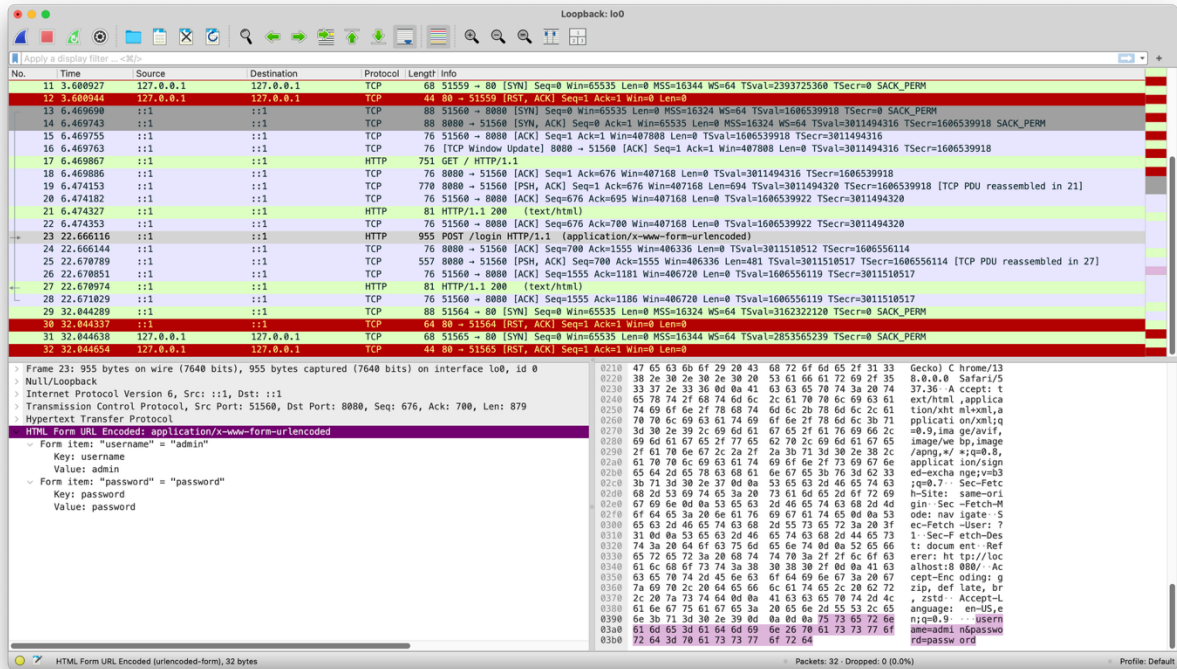


Figure 8 HTTP POST request expanded to show username and password fields in clear text.

Wireshark – Follow HTTP Stream (POST)

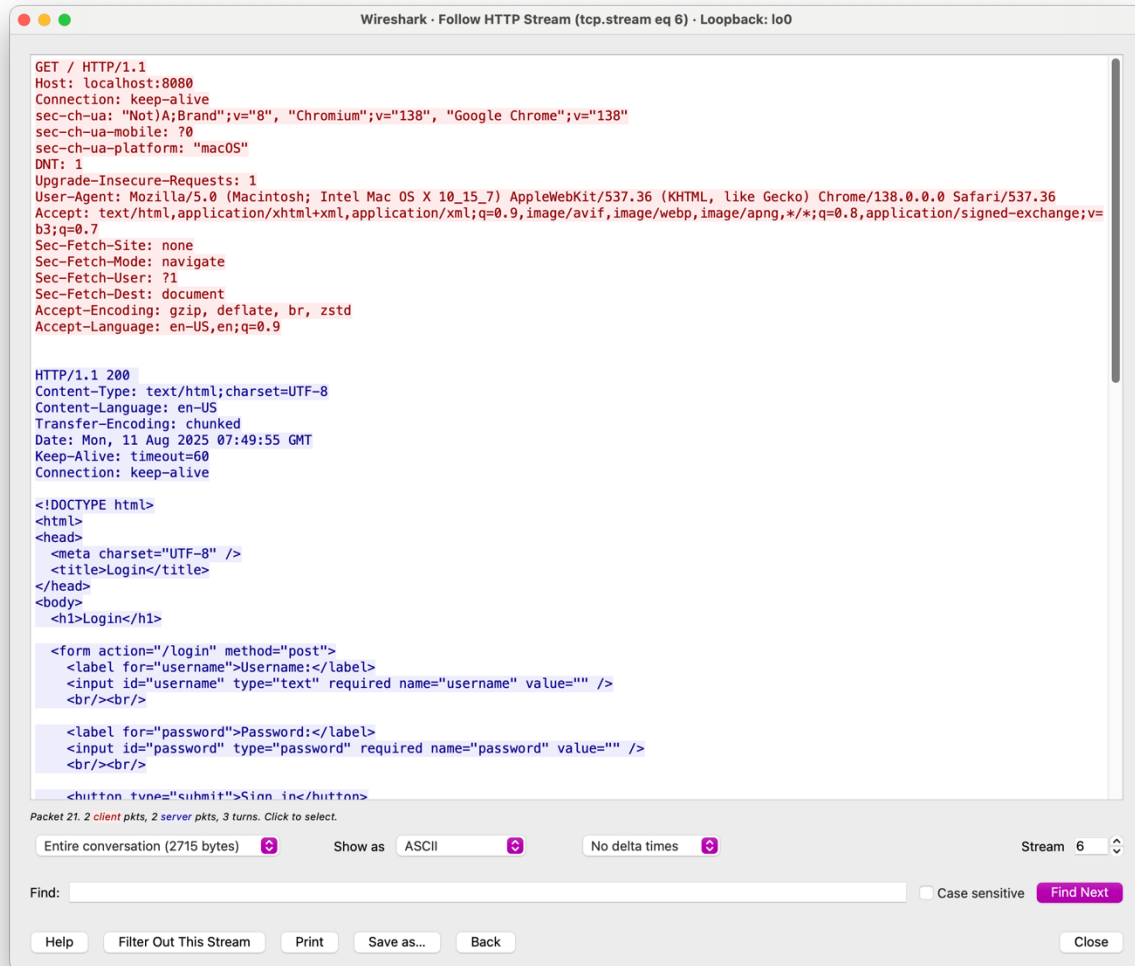


Figure 9 Followed HTTP stream for POST /login showing plain-text credentials and server response.

Transitioning to Secure HTTPS Communication

Screenshots

Browser – HTTPS on 8443 (Warning)

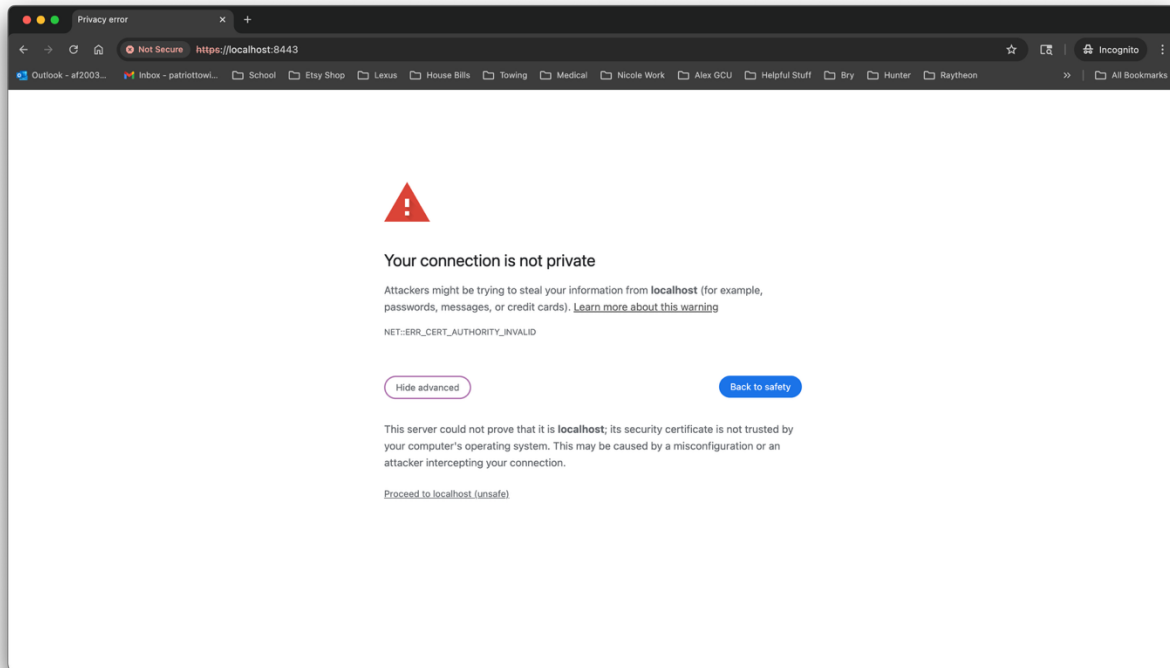


Figure 10 Self-signed certificate warning shown when opening <https://localhost:8443/>.

Browser – Certificate Details (Self-Signed)

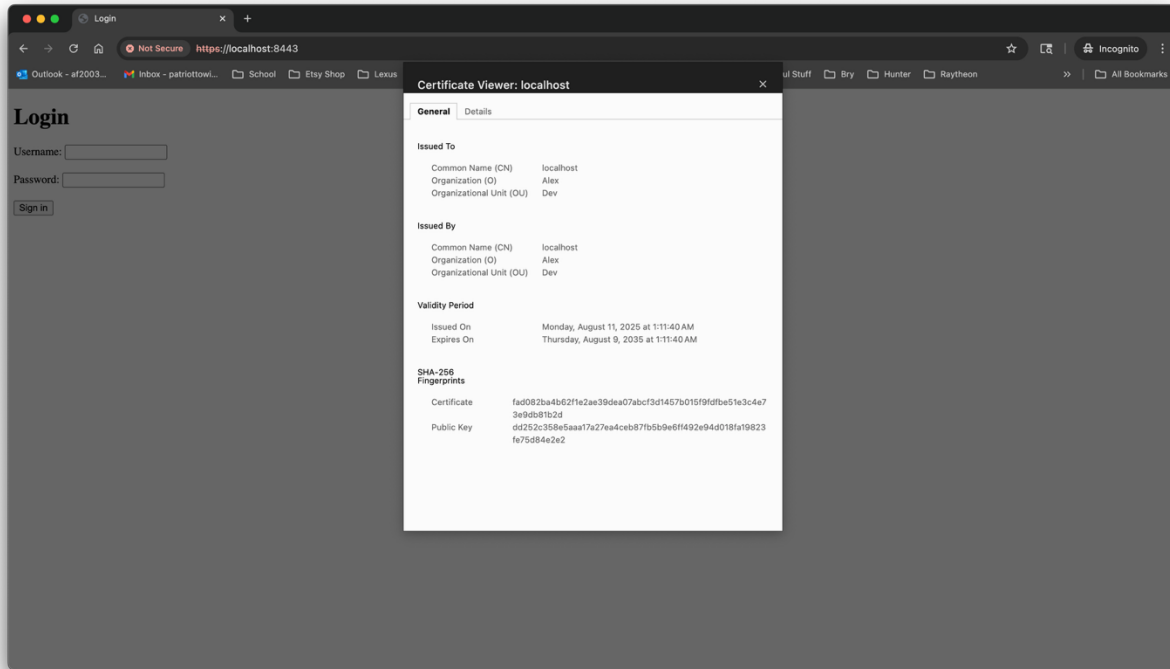


Figure 11 Browser certificate dialog showing a self-signed certificate for `localhost` that is not trusted.

Wireshark – TLS Capture (HTTPS Login)

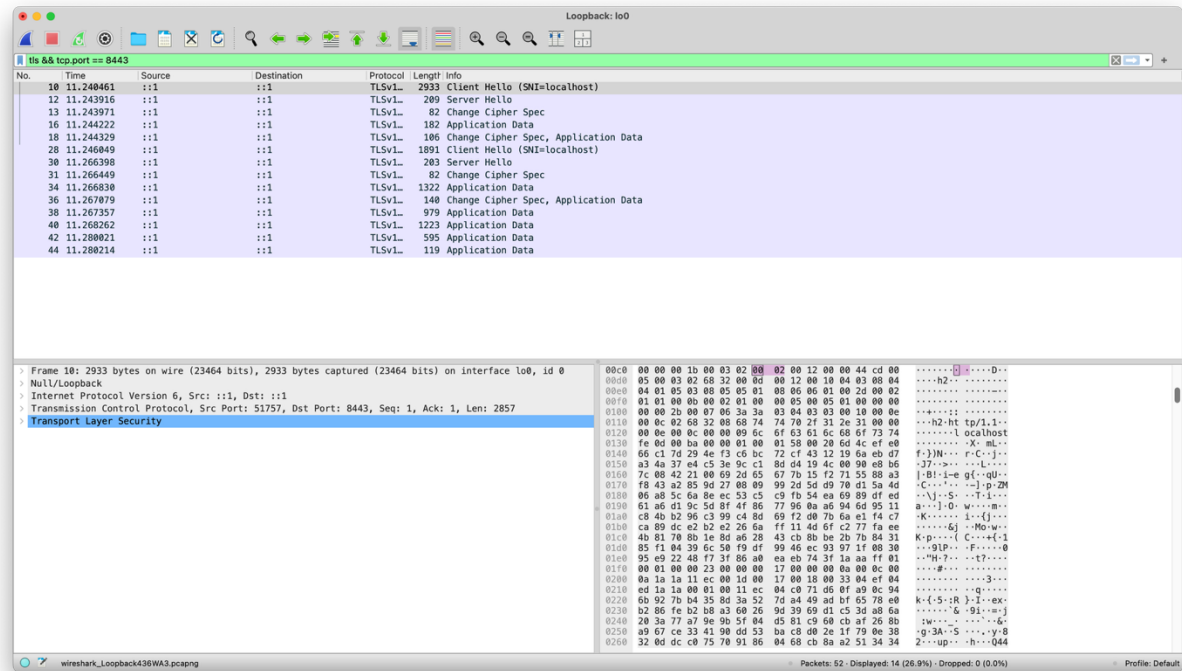


Figure 12 TLS traffic on port 8443 during login; application data is encrypted and HTTP details are not readable.

Summary of Key Concepts

Before this activity, I had never used Wireshark to capture and analyze network traffic, so the entire process was new to me. I started by building and running a simple Spring Boot login application locally. Using Wireshark on the macOS loopback interface, I captured HTTP traffic on port 8080 while submitting login credentials. To my surprise, the POST request clearly displayed the username and password in plain text in the packet details. This made it very clear how vulnerable data is when sent over an unencrypted protocol like HTTP. Even though I understood the general idea that HTTP is insecure, actually seeing the credentials exposed in Wireshark was eye-opening.

Next, I enabled HTTPS on the application using a self-signed certificate and configured it to run on port 8443. When I repeated the login process over HTTPS, the capture in Wireshark looked completely different. Instead of readable HTTP requests and responses, I saw the TLS handshake followed by encrypted Application Data packets. There was no visible sign of the username or password I entered — everything was encrypted. This demonstrated in a very tangible way how HTTPS/TLS protects sensitive information during transmission.

This activity gave me first-hand experience with how network traffic looks in both insecure and secure scenarios. It reinforced the importance of always using HTTPS for login pages and any situation where confidential data is transmitted. The visual proof in Wireshark was more convincing than just reading about the difference between HTTP and HTTPS, and it made me appreciate the value of encryption in everyday web applications.

Other Common Protocols

Protocol	Full Name	Purpose (One Sentence)
TCP	Transmission Control Protocol	Provides reliable, ordered, and error-checked delivery of data over IP networks.
SNMP	Simple Network Management Protocol	Used to monitor and manage network devices remotely.
FTP	File Transfer Protocol	Transfers files between a client and a server over a network.
SMTP	Simple Mail Transfer Protocol	Sends email messages between mail servers.
IMAP	Internet Message Access Protocol	Retrieves and manages email messages directly on the mail server.
POP3	Post Office Protocol version 3	Downloads email messages from a server to a client for offline access.
DNS	Domain Name System	Translates domain names into IP addresses for locating services on a network.
SSH	Secure Shell	Provides encrypted remote login and command execution over an insecure network.
UDP	User Datagram Protocol	Offers fast, connectionless data transmission without guaranteed delivery.
RDP	Remote Desktop Protocol	Enables remote access to a graphical desktop environment over a network.
VoIP	Voice over Internet Protocol	Transmits voice and video communications over IP networks.
DHCP	Dynamic Host Configuration Protocol	Automatically assigns IP configuration settings to network devices.
LDAP	Lightweight Directory Access Protocol	Accesses and manages directory information services.
TLS	Transport Layer Security	Encrypts application-layer network traffic to provide secure communications.